The first one is the *Query Cache*[23] which is a cache of the SQL statements. Query Cache entries only live for the duration of a Rails controller action.

Loading a Rider's first Trip Request at the beginning of an action with `TripRequest.where(rider_id: 1).first` will place the query and result into the Query Cache. The second time the same query is run inside the same controller action, the query will match against the cached version and the result will be returned from cache instead issuing a second database query to PostgreSQL.

In this way, Active Record helps you avoid unnecessary queries with the Query Cache. To see this in action, take note of `CACHE` entries in your Rails server log and execution times printed as `(0.0ms)`. The `CACHE` statements in the Rails log can also be disabled (See: *Disable Rails CACHE Logging*[24]).

A trade-off with the Query Cache is that more memory is used hold the query and the result. Memory is used to instantiate Active Record objects and their relationships populated from the Query Cache (See: *Cached queries guidelines*[25]).

While being a useful feature, an over reliance on the Query Cache usage from excessive memory consumption could be an opportunity to improve your database queries making them faster and lighter.

In the next section you'll see PostgreSQL Prepared Statements[26] and how to work with them in Active Record.

Prepared Statements With Active Record

Prepared Statements[27] are a type of object in PostgreSQL that take a specific query and separate the parameters that are bound to it.

Prepared Statements give a name to the query without parameters. Prepared statements can be viewed by querying the system view `pg_prepared_statements`. Try running `SELECT * FROM pg_prepared_statements;` from psql to see what's stored in there.

Prepared Statements are enabled by default and store up to 1000 statements per connection with Active Record.

23. https://guides.rubyonrails.org/caching_with_rails.html#sql-caching
24. https://heliom.ca/blog/posts/disable-rails-cache-logging
25. https://docs.gitlab.com/ee/development/cached_queries.html
26. https://www.postgresql.org/docs/current/sql-prepare.html
27. https://guides.rubyonrails.org/configuring.html